

FY-26: Career Preview Program project Review (CPP-3)

Updated June 2026

College /University Name: Jaipur Engineering College & Research Centre

AGENTIC BUG TRIAGE & ROUTING

Project Name: Agentic Bug Triage & Routing

College / University Name: Jaipur Engineering College & Research Centre

Project Participants : Anuj Modani
Pulkit Jain
Shivansh Gaur
Disha Jain
Om Jain

Mentor Names : Madhukar Rupakumar, Divakar Padiyar, PE Rajesh, Bharath Narayan (HPE)
Aneesh Kumar Mishra (College Mentor)

GitHub Link :

Source Code: <https://github.com/pulkitjn3010/agentic-bug-triage-and-routing>
Project Documents: <https://github.com/sh1vanshgaur/Agentic-Bug-Triage>



Problem Demonstration

STUCK IN THE SEARCH LOOP

Jira **GitHub** **Bugzilla** **Confluence**

Spark History Server Fails to Bootstrap

Overview

Apache Spark does not automatically create the `spark.history.fs.logDirectory` or `spark.eventLog.dir` directories. If either directory does not exist before the History Server or application starts, Spark will fail silently or throw an error. This is intentional — Spark's `Utils.createDirectory()` explicitly does not mark directories for automatic creation or deletion.

Affected Configuration Properties

`spark.history.fs.logDirectory` — the directory the History Server scans to reconstruct completed application UIs. Must exist before starting the History Server.

`spark.eventLog.dir` — the directory where running applications write event logs. Must exist before `spark.eventLog.enabled=true` takes effect.

Common Error Patterns



Problem Demonstration : <https://youtu.be/gOysplhLMJc>

Problem Statement

1. Multi-System Data Fragmentation

Bug data is spread across multiple systems with no unified view

2. Manual Context Gathering

Engineers manually search tickets, logs, and customer cases.

3. Missing Cross-System Correlation

Related bugs across systems are not linked automatically.

4. Scattered Knowledge Repositories

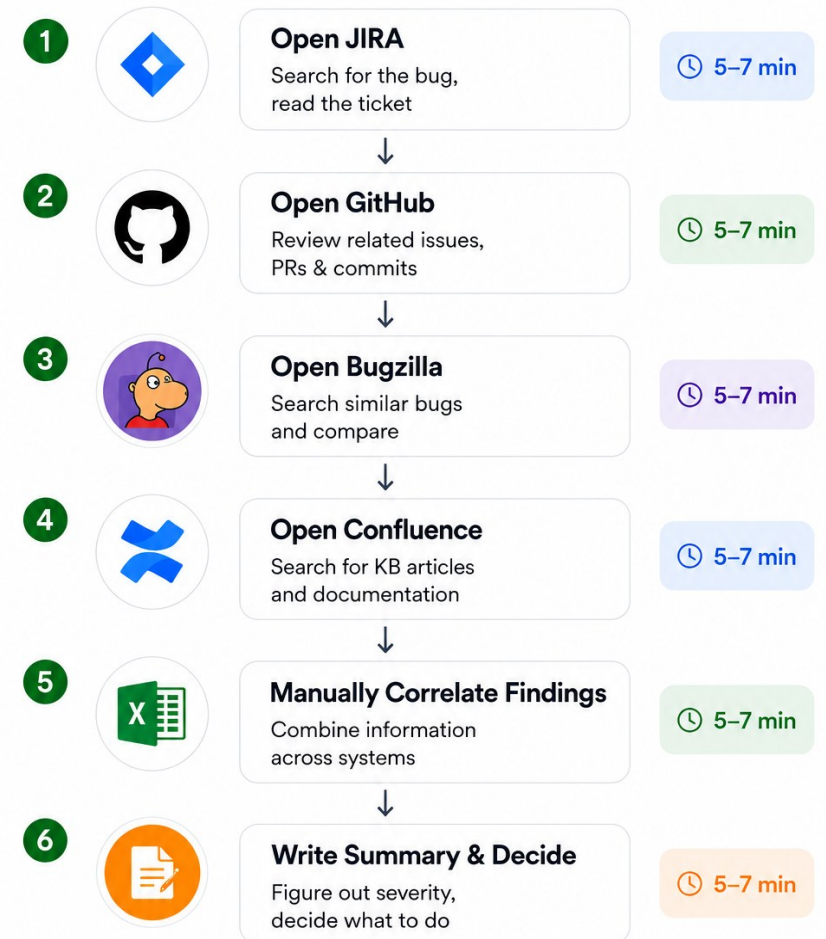
Runbooks and workarounds remain buried in knowledge bases.

5. Lack of Actionable Triage Intelligence

Trackers provide no AI-driven severity, root cause, or recommendations.

The Reality of Enterprise Bug Triage

An engineer triaging a critical production bug today.



This process takes
30-40 minutes
per bug

As the number of active incidents grows, manual context gathering becomes **increasingly difficult**.

Time spent gathering context — *before* fixing the bug.

Solution Overview

1. Unified Bug View

Bring Jira, GitHub, Bugzilla and Confluence into one workspace.

2. Automated Context Gathering

Collect bug details, metadata and logs automatically.

3. Cross-System Correlation

Related bugs across systems are linked automatically.

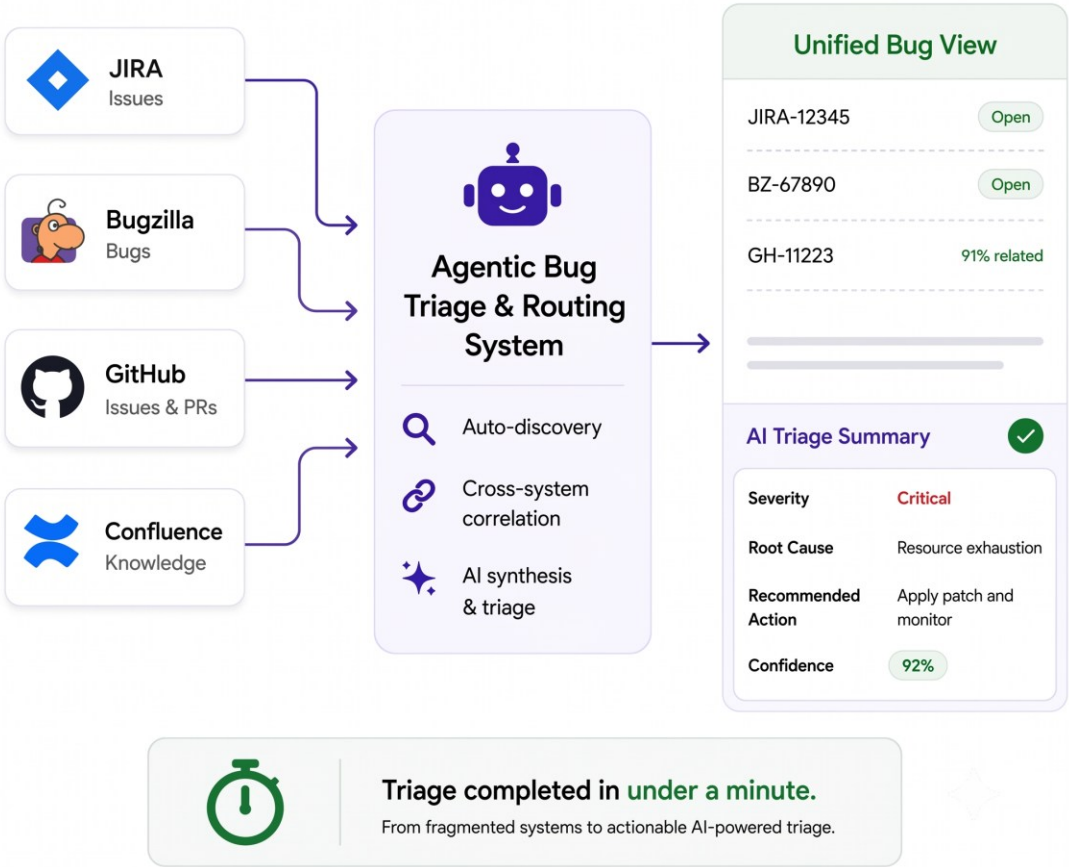
4. Knowledge Base Enrichment

Retrieve relevant runbooks, documentation and historical fixes.

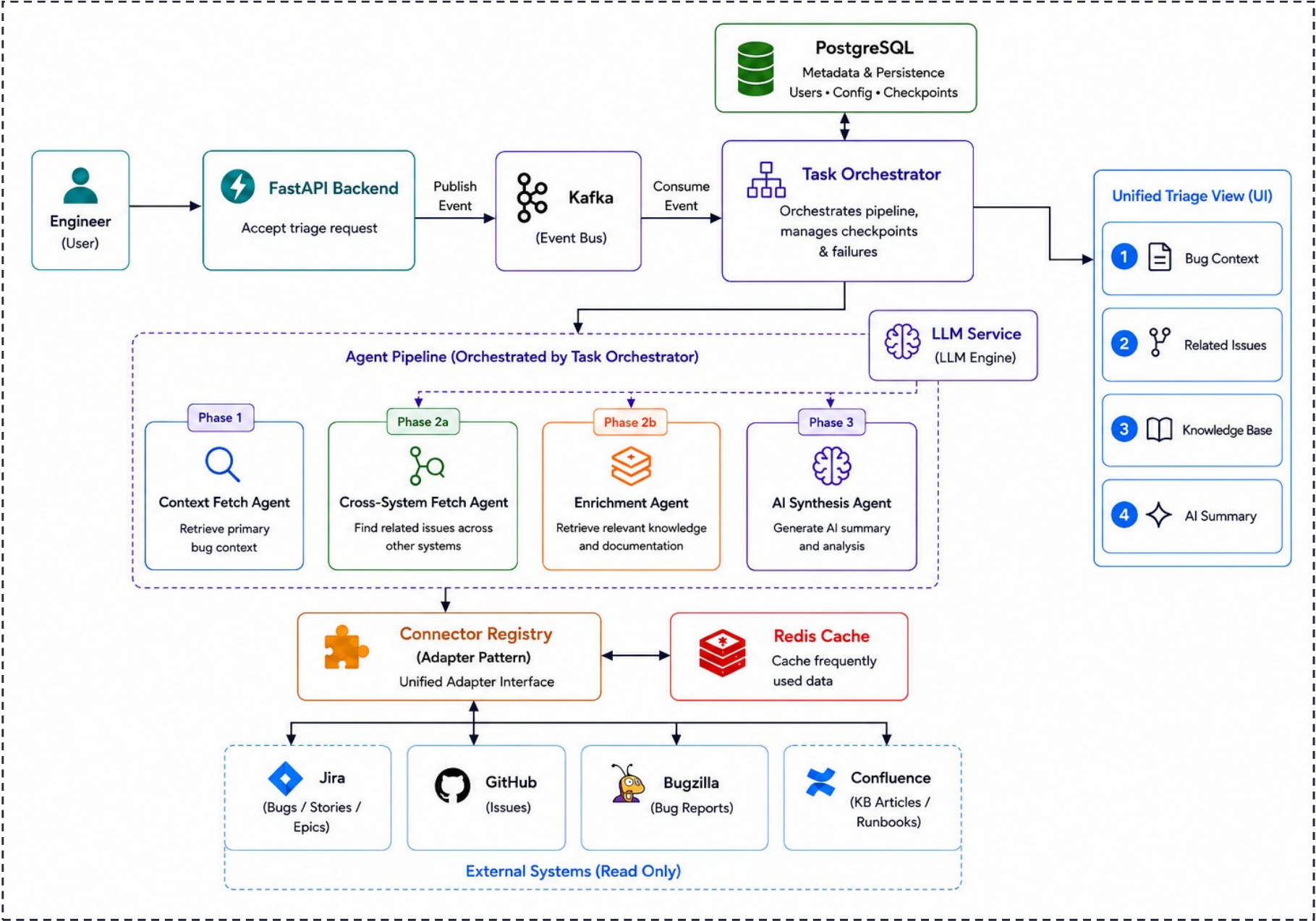
5. AI Powered Triage

Generate severity, root cause, confidence score and recommended actions.

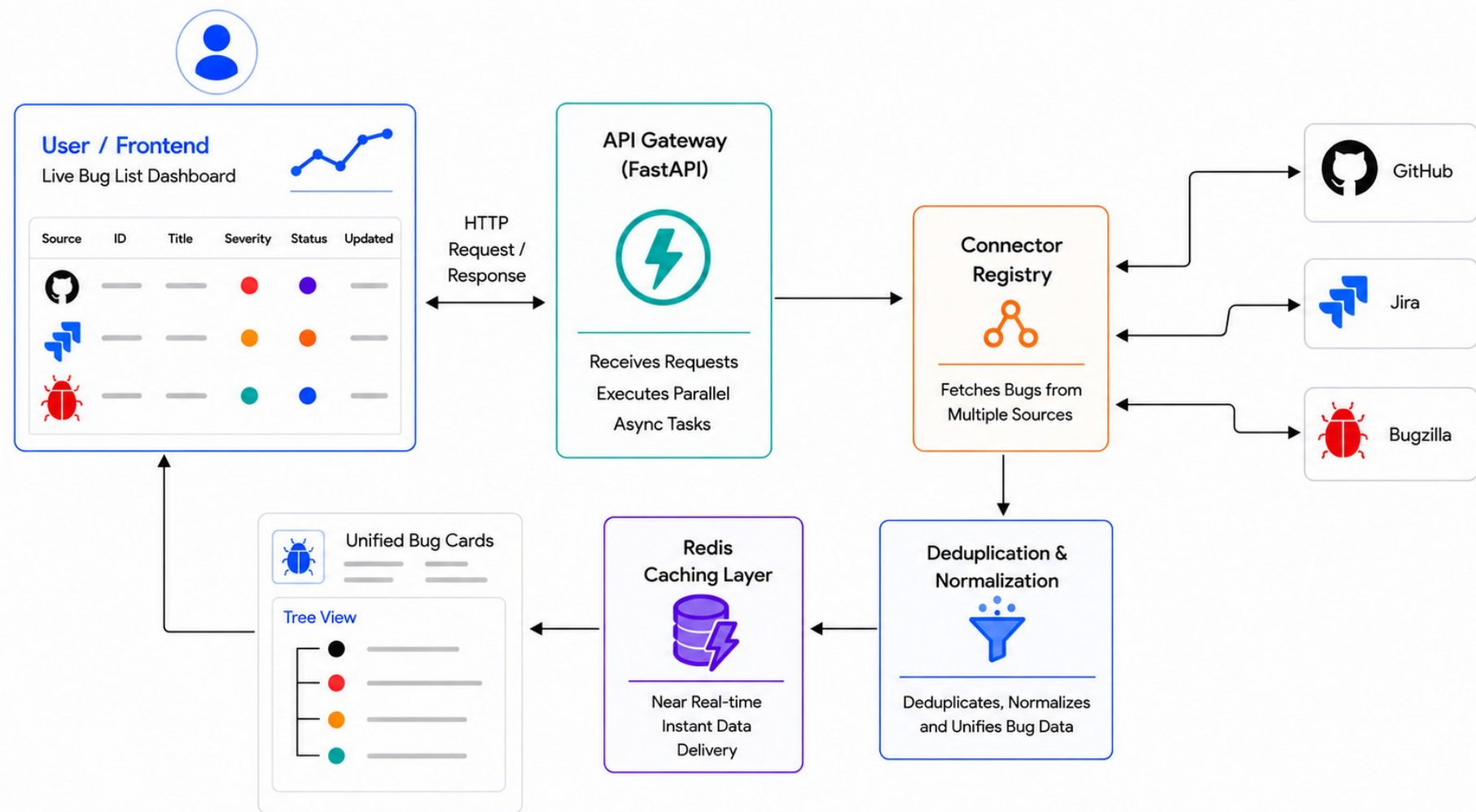
Connect your bug tracking systems, auto-discover related bugs, and generate AI-powered triage—all from a **single unified workspace**.



System Architecture



Live Bug Discovery Pipeline



Parallel Fetching

- Async API calls
- Multi-source aggregation

In memory filtering

- Instantaneous search and filtering

Dashboard

- Live bug updates Unified bug cards

Tree View

- Related bug discovery across multiple tracking systems.

Unified Bug List View

1 At-a-glance Summary

See priority counts, triage stats, and system health instantly.

2 Smart Filters

Filter by project, source, severity, or status. Updates instantly from cache.

4 Expanded Tree View

Click to expand and view all related bugs across systems under a single root issue.

3 Consistent View

Unified badges and metadata across Jira, GitHub, and Bugzilla.

PQ: 84

P1: 4472

☒ Triage Today: 3

☒ Needs Triage: 5889

Failed: 0

2/2 Systems Online

Auto-Discovered Bugs

5892 bugs · fetched live · Redis cache 2 min TTL · nothing stored

Search by ID, title, keyword

All Projects

All Sources

All Severities

All Statuses

All

Untriaged

Triaged

Critical

Enter bug ID to triage directly...

Triage

BT-001

Root / Primary

JIRA

SPARK-28869

Switch log directory from Spark UI without restarting history server

P2

88%

Open

5 Related

Triage expired

Re-triage

Related

JIRA

SPARK-56966

History server fails to start when spark.history.fs.logDirectory does not exist; auto-create the directory

Unknown

open

⌵

Related

JIRA

SPARK-37639

spark history server clean event log directory with out check status file

Unknown

open

⌵

Related

GH

#55975

Automatically create the spark history log directory

Unknown

open

⌵

Related

JIRA

SPARK-27523

Resolve scheme-less event log directory relative to default filesystem

Unknown

open

⌵

Related

JIRA

SPARK-48575

spark.history.fs.update.interval calling too many directory pollings when spark log dir contains many sparkEvent apps

Unknown

open

⌵

BT-003

Root / Primary

GH

55747

Is Spark limited to split the Parquet read granularity by Row Group level only?

P2

88%

Open

3 Related

View Previous Results

Related

JIRA

SPARK-56871

Split parquet reads more granularly

Unknown

open

⌵

Related

GH

#56811

[SPARK-57415][SQL] Parquet vectorized reader performance improvements (umbrella)

Unknown

open

⌵

Related

JIRA

SPARK-57415

Parquet vectorized reader performance improvements

Unknown

open

⌵

JIRA

SPARK-57624

from_xml to variant should honor the parse mode

P8

Open

38/6/2026

Triage

JIRA

SPARK-57343

[SECURITY] Upgrade bundled Netty to 4.2.15.Final and ZooKeeper to 3.9.5 in PySpark to resolve Critical/High CVEs

P8

Open

38/6/2026

Triage

JIRA

SPARK-58110

Fix 'from_csv': parse fails when data contains spaces before and after

P8

Open

29/6/2026

Triage

JIRA

SPARK-54137

Prepare Apache Spark 4.2.0

P8

Open

25/6/2026

Triage

JIRA

SPARK-56488

Upgrade Scala 2.13 in 3.5 branch for CVE-2022-36944

P8

Open

25/6/2026

Triage

JIRA

SPARK-55330

Unable to create PVC through Spark 4.1.0

P8

Open

24/6/2026

Triage

JIRA

SPARK-56754

Prepare Apache Spark 4.3.0

P8

Open

19/6/2026

Triage

JIRA

SPARK-45154

Pyspark DecisionTreeClassifier: results and tree structure in spark3 very different from that of the spark2 version on the same data and with the

P8

Open

17/6/2026

Triage

Previous

1

2

3

...

590

Next

Go to:

5 One-Click Triage

Instantly triage any bug with a single click.

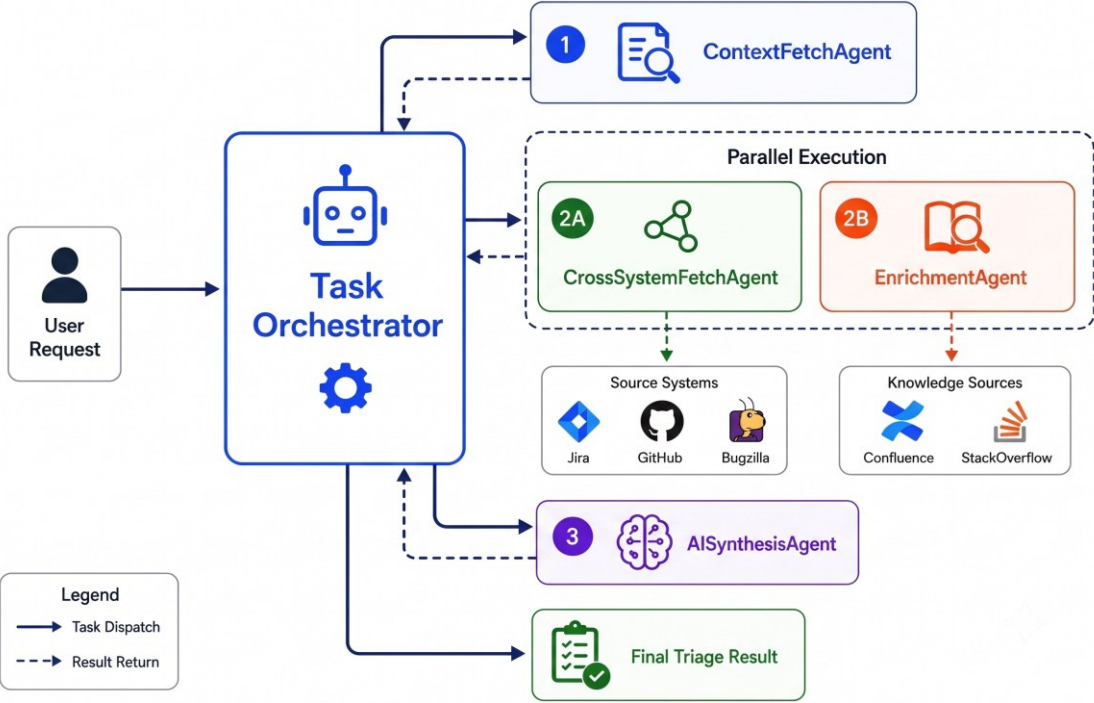
6 Priority & Actions

Clear priority badges with quick actions to triage any bug.

Pipeline Coordination & Agent Roles

TASK ORCHESTRATOR

- Starts the triage pipeline on every bug request
- Validates inputs and initializes shared pipeline context
- Sequences and triggers agents in the correct order
- Stores checkpoints after each agent stage
- Handles agent failures without breaking the pipeline
- Publishes final triage results to the UI



1

ContextFetchAgent

Input	bug_id, source_id, case_id
Output	primary_ticket
Purpose	Fetches primary bug details

2

CrossSystemFetchAgent

Input	primary_ticket
Output	related_tickets
Purpose	Finds related bugs across systems

3

EnrichmentAgent

Input	primary_ticket
Output	kb_articles
Purpose	Finds troubleshooting evidence

4

AISynthesisAgent

Input	primary_ticket + related_tickets + kb_articles
Output	ai_summary
Purpose	Combines all context into final triage result

Unified Triage View

01 Bug Context ⓘ

JIRA

Ticket ID

SPARK-28869

Source

Apache Spark (JIRA)

Title

Switch log directory from Spark UI without restarting history server

Status

Open

Severity

P3

Component

Web UI

Reporter

Noritaka Sekiyama

Updated

3/17/2020, 4:20:38 AM

Created

6/17/2019

URL

Open source ticket

DESCRIPTION

History server polls the directory specified in spark.history.fs.logDirectory, and displays the event logs based on the data included in the directory. In the Cloud, event logs might be written into different directories from different clusters. Currently it is not so easy to launch history server for multiple clusters because there is no ways to switch log directory after starting the history server. If users can switch log directory from Spark UI without restarting history server, it woul...

02 Related Issues ⓘ

5 found

JIRA

SPARK-27523

Resolve scheme-less event log directory relative to default file...

Open

/

80% Very Similar

The candidate discusses a related issue with event log directories and the default filesystem.

+ View description

GitHub

#55975

Automatically create the spark history log directory

Open

/

80% Very Similar

The candidate discusses a similar issue with the history log directory and its creation.

- Hide description

If the directory "spark.history.fs.logDirectory" does not exist, the history server will fail to start. Automatically creating this directory can reduce errors.

Similarly, if the spark.eventLog.dir does not exist, the operation will also fail, even if it is written to S3.

JIRA

SPARK-48575

spark.history.fs.update.interval calling too many directory polli...

Open

/

80% Very Similar

03 Knowledge Base ⓘ

1 results

high

cannot start spark history server

I am running spark on yarn cluster. I tried to start the history server ./start-history-server.sh but got the following errors. starting org.apache.spark.de...

04 AI Summary ⓘ

80% confidence

P2

80% AI confidence score

ROOT CAUSE

Inability to switch log directory from Spark UI without restarting history server

RECOMMENDED ACTIONS

01. Implement a feature to dynamically update the log directory in the Spark UI

02. Modify the history server to poll the new log directory without requiring a restart

03. Add input validation to ensure the new log directory exists and is accessible

AFFECTED COMPONENTS

Web UI

History Server

SUMMARY

The Spark UI does not allow users to switch the log directory without restarting the history



Connector Configuration Dashboard

1

System Filter

Quickly filter connectors by Jira, GitHub, Apache JIRA, Bugzilla, Confluence, Support KB, etc.

2

Connector Card

Each card shows connector details including name, endpoint, project/space, access type and status.

Connections

Manage source system connectors

FILTER BY SYSTEM

All Systems7

JIRA0

GitHub3

Apache JIRA2

Bugzilla1

Confluence1

Support KB0

All Systems

Connectors seeded via init_db.py on startup. Changes take effect immediately.

3

4

5

GH

Apache Spark (GitHub)

https://api.github.com/apache/spark

Access: Authenticated

Status: Connected

Test

Edit

Disable

J

Apache Spark (JIRA)

https://issues.apache.org/jira/SPARK

Access: Authenticated

Status: Connected

Test

Edit

Disable

GH

Apache Kafka (GitHub)

https://api.github.com/apache/kafka

Access: Authenticated

Status: Disconnected

Test

Edit

Enable

J

Apache Kafka (JIRA)

https://issues.apache.org/jira/KAFKA

Access: Authenticated

Status: Disconnected

Test

Edit

Enable

BZ

Mozilla Firefox (Bugzilla)

https://bugzilla.mozilla.org/Firefox

Access: Authenticated

Status: Disconnected

Test

Edit

Enable

GH

Kubernetes — GitHub Issues

https://api.github.com/kubernetes/kubernetes

Access: Authenticated

Status: Disconnected

Test

Edit

Enable

CF

HPE Engineering KB (Confluence)

https://cpp3-hpe.atlassian.net/wiki/HPEKB

Access: Authenticated

Status: Connected

Test

Edit

Disable

6

Add Connector

Add a new connector (integration) dynamically without restarting the application.

3

Test Connection

Perform a live health check to verify API endpoint availability and validate credentials.

4

Edit Configuration

Edit connector configuration such as endpoint URL, project/space key, and authentication details.

5

Enable / Disable

Enable or disable connectors instantly. Useful during maintenance or to control data syncing.

Audit Log & Data Sovereignty

- **Zero-Residual Data Policy**
 - Raw bug content are held temporarily in pipeline context only
 - All raw data is purged immediately on run completion or failure
- **Metadata-Only Audit Trail**
 - Persistent logs capture only operational metadata: user ID, execution time, connectors queried
 - Raw bug texts and files are never written to the audit store.
- **Anonymized LLM Output**
 - Only high-level classifications are retained: unified severity (P0–P3), confidence score (0.0–1.0), truncated root cause hypothesis.
 - No raw LLM reasoning or source content is persisted.

Triage History ⓘ								
Recent pipeline completions · last 50								
TRiage ID	BUG ID	SOURCE	SEVERITY	CONFIDENCE	ROOT CAUSE	DURATION	TRIAGED AT	ACTIONS
BT-005	SPARK-44988	JIRA	P0	80%	INT64 (TIMESTAMP(NANOS,false)) ...	34.5s	Jul 2, 2026 · 10:17	View Results Re-triage
BT-004	SPARK-28069	JIRA	P2	80%	Inability to dynamically switch log ...	35.9s	Jul 2, 2026 · 10:15	Triage expired Re-triage
BT-003	SPARK-57343	JIRA	P0	95%	Outdated and vulnerable versions ...	34.5s	Jul 2, 2026 · 09:50	Triage expired Re-triage



AGENTIC BUG TRIAGE AND ROUTING SYSTEM



 **Live Demo:** <https://youtu.be/MUy3MSw08wI>

Key Learnings

1. Multi-Agent Architecture

Learned how to decompose complex workflows into specialized AI agents coordinated by a central orchestrator.

2. Unified Bug Discovery

Built a live discovery pipeline with parallel fetching and intelligent Redis caching for faster responses.

3. LLM Integration

Implemented robust LLM integration using validation, structured outputs, and fallback handling.

4. Enterprise Data Sovereignty

Designed a privacy-first architecture that preserves data sovereignty across enterprise systems.

5. Adapter Design

Applied the Adapter Pattern to integrate multiple enterprise platforms through a unified interface.

6. Reliability & Fault Tolerance

Enhanced system reliability using checkpointing, caching, and resilient execution strategies.



Future Scope

1. AI-Powered Semantic Correlation

Improve duplicate detection and related bug discovery using semantic embeddings.

2. Intelligent Role-Based Dashboards

Deliver personalized dashboards and AI insights tailored to different organizational roles.

3. Secure Credential Management

Centralize connector authentication with enterprise-grade security and access control.

4. Enterprise Connector Expansion

Integrate additional enterprise platforms through the scalable Connector Registry.

5. On-Premises AI Deployment

Enable air-gapped deployments using locally hosted LLMs for regulated environments.



Conclusion

Unified Bug Intelligence Across Enterprise Systems

Connected JIRA, GitHub, Bugzilla, and Confluence into a single unified workspace.

From Manual Investigation to Intelligent Automation

Automated context gathering, cross-system correlation, knowledge enrichment, and AI-assisted triage.

One-Click End-to-End Bug Triage

Delivered unified bug context, related issues, supporting knowledge, and AI recommendations in under a minute.

Enterprise-Ready Architecture

Scalable, extensible, read-only by design, with plug-and-play connector integrations.



Thank You



Tech Stack & Architecture Ecosystem

1. Backend & API Layer - Python 3.11, FastAPI, WebSockets

Purpose: Powers asynchronous APIs and enables real-time communication between the frontend and backend.

2. Async Execution Layer - Apache Kafka, Asyncio

Purpose: Supports event-driven workflows and parallel execution of AI agents.

3. Data & Caching Layer - PostgreSQL, Redis

Purpose: Stores application metadata, connector configurations, and accelerates performance through caching.

4. AI & Reasoning Layer - Groq API, Llama 3 70B

Purpose: Performs semantic understanding, bug correlation, severity prediction, and AI-assisted reasoning.

5. Frontend Layer - React, Vite, Tailwind CSS

Purpose: Provides an interactive dashboard for live bug discovery, monitoring, and triage visualization.

6. Enterprise Integrations - Jira, GitHub, Bugzilla, Confluence

Purpose: Retrieves live bug reports and knowledge-base information from enterprise platforms.

7. Deployment Layer - Docker, Docker Compose

Purpose: Enables containerized, portable, and reproducible deployment across environments.



Team Contribution

Pulkit Jain: API Gateway, Authentication & Authorization, REST APIs, Service Layer, WebSocket Communication

Shivansh Gaur: PostgreSQL Database Layer, Kafka Event Processing, Redis Caching, Docker & Infrastructure Setup

Disha Jain: Context Fetch Agent, AI Synthesis Agent, LLM Integration, Pipeline Orchestration

Anuj Modani: Cross-System Fetch Agent, Similarity Search, Knowledge Enrichment Agent, Related Issue Correlation

Om Jain: JIRA Integration, GitHub Integration, Bugzilla Integration, Confluence Integration, Connector Registry & Adapter Pattern

